

ABM Platform — Final Architecture (V1, Locked)

Status: Canonical architecture, decision-locked. No more base-level rewrites.

Owner: Sunny Dsouza • **Primary user:** Galyna • **Date:** May 22, 2026

Supersedes: V1_VISION.md and V1_PRODUCTION_VISION.md (historical)

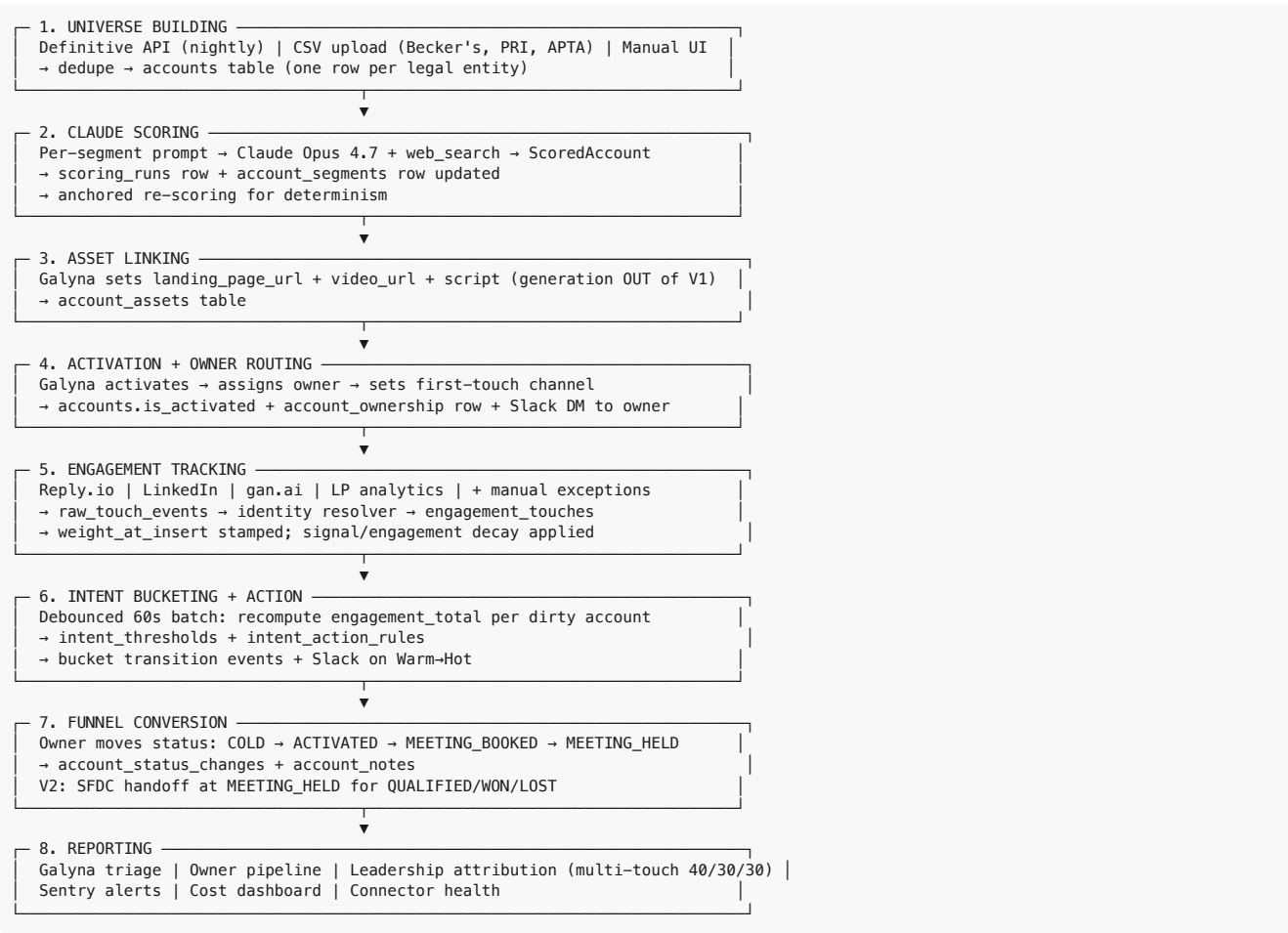
§0 — TL;DR

This is the locked architecture for Magical's ABM Intelligence Platform V1. Every base decision (entity model, data flow, ingestion shape, engagement computation, status chain, failure modes, scaling pattern) is decided. Galyna has 9 workflow questions remaining (see §9); the engineering shape will not change as a result of her answers.

The platform is a **single-tenant FastAPI + Next.js + Postgres + Redis monorepo on Railway**, running an **8-stage pipeline**: ingest → score → asset-link → activate → engage → bucket → convert → report. Scoring uses Claude Opus 4.7 + web_search with **anchored re-scoring** for determinism.

Engagement uses **multi-stage ingestion with identity resolution** and **debounced 60s batch recomputation** to handle bursty webhook traffic from Reply.io and similar tools. Every architectural risk identified in critical review (15 flaws) is either fixed in V1 or has a documented deferral with a clear owner and trigger condition.

§1 — The 8-stage pipeline (high level)



§2 — The cast (personas and what they do)

GALYNA – analyst, primary power user • Activates accounts (only she does this) • Tunes channel weights, intent thresholds, decay half-lives • Manages suppression list • Reviews triage view Monday morning • Resolves identity-resolution queue weekly • Reviews fuzzy-merge candidates
ACCOUNT OWNERS – SDRs / AEs Justin, Stephen, Aidan, Tyler, Matt, Colin

- Receive activated accounts
 - Add notes, move status forward
 - Get pinged on Warm-Hot bucket changes
 - V1: see all accounts, default filter "my accounts"
- LEADERSHIP – Geoffrey, Marco, Harpaul
- Read attribution dashboard
 - Cost per scored account, win rate by source, ARR attribution
- PLATFORM (background workers)
- Nightly Definitive pull
 - Hourly engagement connector polls
 - Identity resolver every 5 min
 - Engagement recompute every 60s for dirty accounts
 - Auto re-score on signal for Tier 1/2 accounts
 - Bucket transition Slack DMs

§3 — Account lifecycle (the complete post-scoring journey)

T+0	SCORING RUN COMPLETES • scoring_runs row inserted with structured_json + version stamp • account_segments row created/updated (segment, score, tier) • effective_tier surfaced via SQL view (no cache) • audit_log: "scored" • Account still is_activated=FALSE • Surface: Galyna's Triage view only
T+0...T+7d	COLD QUEUE • Auto re-score on new signal (Tier 1/2 only, rate-limited) • Galyna can manually re-score anytime • Suppressed accounts: scored & tracked but never activated/alerted • PARSE FAILURE: prior score preserved; UI flags "last failed"
T+7d	ACTIVATION GATE Galyna clicks Activate. Atomic transaction: • accounts.is_activated = TRUE • account_ownership row inserted (owner=Colin, assigned_at=now) • accounts.channel_first_touch = "TOFU" • account_status_changes: COLD → ACTIVATED • audit_log ASYNC: Slack DM to Colin; asset check (LP/video URLs present?)
T+7d→	ENGAGEMENT ACCRUAL (always-on) Each connector (Reply.io, LinkedIn, gan.ai, LP analytics) polls hourly: [Source] → raw_touch_events (always inserted, never lost) <pre> graph TD A[raw_touch_events] --> B{identity resolver every 5 min} B -- matched --> C[engagement_touches weight_at_insert stamped] B -- unmatched --> D[review queue for Galyna] </pre> Each engagement_touches insert marks account as "dirty"
T+7d→	INTENT BUCKETING (debounced) Every 60s, batch job processes dirty accounts: • Recompute engagement_total with decayed touch weights • Compare to intent_thresholds → determine bucket • If unchanged: UPDATE state, no side effects • If changed UP: UPDATE state, INSERT account_bucket_transitions → Slack DM to Owner (Warm-Hot triggers) → If last scoring run >7d old: enqueue background re-score (rate limited) • If changed DOWN: UPDATE state, no DM
T+8d	OWNER FIRST OUTREACH (manual) Colin opens Account Detail. Reads one-pager. Crafts email in Reply.io. Reply.io send event → poll picks up → engagement_touches row. PLATFORM DOES NOT SEND EMAILS in V1. Decision support, not automation.
T+10d	RESPONSE Matt replies. Reply.io records event. → engagement_touches(channel='Response_Agreed', count=1, weight=10) → engagement_total likely crosses Hot threshold Colin manually moves status: ACTIVATED → MEETING_BOOKED • account_status_changes row • audit_log • Optional Slack to Galyna if she's watching
T+14d	MEETING_BOOKED → MEETING_HELD Colin updates status. Adds note: "Strong fit, sending proposal"
T+45d (V1)	POST-MEETING TRACKING (manual notes only) V1 status chain ends at MEETING_HELD. There is no QUALIFIED / WON status in V1. Post-meeting outcomes (proposal sent, won, lost) are captured in account_notes as free text by the Owner. • Colin writes a note: "Sent proposal Mar 10. Verbal yes Mar 22. ARR ~\$180K." • is_customer flag toggled manually on close-won by Galyna or Owner • Attribution events derived from notes + status history when reporting runs

```
T+45d (V2)  SFDC HANDOFF (when sync ships)
            At MEETING_HELD, system auto-creates SFDC Opportunity.
            Post-MEETING_HELD statuses (QUALIFIED, PROPOSAL, WON, LOST) pulled from SFDC.
            ARR + close metadata read from SFDC.
            V1's manual notes become input fodder for SFDC's Opportunity record.
```

§4 — The complete data model

§4.1 Core entities

```
-- THE COMPANY (one row per legal entity, even if multi-segment)
CREATE TABLE accounts (
  id          BIGSERIAL PRIMARY KEY,
  name        TEXT NOT NULL,
  normalized_name TEXT NOT NULL,      -- for dedupe (lowercase, suffix-stripped)
  domain      TEXT,                  -- primary dedupe key
  ein         TEXT,                  -- tax ID if known (most reliable)
  is_customer BOOLEAN NOT NULL DEFAULT FALSE,
  is_activated BOOLEAN NOT NULL DEFAULT FALSE,
  channel_first_touch TEXT,          -- TOFU | BOFU | Email | Event | Other
  current_status TEXT NOT NULL DEFAULT 'COLD',
  -- COLD | ACTIVATED | MEETING_BOOKED | MEETING_HELD | PARKED
  -- (V2: + QUALIFIED | WON | LOST_PERMANENT | LOST_RE_ENGAGEABLE via SFDC)
  parked_until DATE,                -- auto-revive from PARKED on this date
  is_suppressed BOOLEAN NOT NULL DEFAULT FALSE,
  suppression_reason TEXT,
  suppressed_until DATE,
  suppressed_by_user_id BIGINT,
  suppressed_at         TIMESTAMPTZ,
  version               INTEGER NOT NULL DEFAULT 1, -- optimistic locking
  first_seen_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  source                TEXT NOT NULL,             -- 'definitive' | 'manual' | 'cold_list' | ...
  source_list           TEXT,                      -- "Definitive nightly 2026-05-23", "Cold Ortho", etc.
  UNIQUE(domain),      -- domain is primary dedupe key
  UNIQUE(normalized_name) -- secondary dedupe (when domain missing)
);

-- SEGMENTS scored for an account (1 row per scored segment per account)
CREATE TABLE account_segments (
  id          BIGSERIAL PRIMARY KEY,
  account_id  BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  segment     TEXT NOT NULL CHECK (segment IN ('specialties','payer','hs')),
  current_score NUMERIC(5,2),        -- denorm cache of latest run
  current_tier TEXT,                 -- denorm cache (overridden in view)
  current_max  INTEGER NOT NULL,     -- 30 or 27 depending on segment
  last_scored_at TIMESTAMPTZ,
  UNIQUE(account_id, segment)
);

-- CONTACTS (one per person per account; portable history when person moves)
CREATE TABLE contacts (
  id          BIGSERIAL PRIMARY KEY,
  account_id  BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  name        TEXT NOT NULL,
  role        TEXT NOT NULL,
  email       TEXT,
  linkedin_url TEXT,
  is_primary  BOOLEAN NOT NULL DEFAULT FALSE,
  source      TEXT NOT NULL,
  observed_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  ended_at    TIMESTAMPTZ,          -- set when person leaves the account
  prior_contact_id BIGINT REFERENCES contacts(id) -- chain for career history
);

-- ASSETS per account
CREATE TABLE account_assets (
  id          BIGSERIAL PRIMARY KEY,
  account_id  BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  asset_type  TEXT NOT NULL CHECK (asset_type IN
    ('landing_page','video','email_template','embed','other')),
  url         TEXT,
  script_or_copy TEXT,
  embed_code  TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  source      TEXT NOT NULL DEFAULT 'manual',
  UNIQUE(account_id, asset_type)
);
```

§4.2 Scoring

```
CREATE TABLE scoring_runs (
  id          BIGSERIAL PRIMARY KEY,
  account_id  BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  segment     TEXT NOT NULL,
  total_score NUMERIC(5,2) NOT NULL,
```

```

max_score      INTEGER NOT NULL,
tier           TEXT NOT NULL,
score_band_low NUMERIC(5,2),           -- for displaying "25 (24-26 range)"
score_band_high NUMERIC(5,2),
raw_markdown   TEXT NOT NULL,
structured_json JSONB NOT NULL,
structured_json_version TEXT NOT NULL DEFAULT 'v1.0', -- schema versioning
model          TEXT NOT NULL,
cost_usd       NUMERIC(8,4),
input_tokens   INTEGER,
output_tokens  INTEGER,
stop_reason    TEXT,
parse_failed   BOOLEAN NOT NULL DEFAULT FALSE,
triggered_by   TEXT NOT NULL,         -- manual | signal | scheduled | bucket_change
triggering_signal_id BIGINT REFERENCES signals(id),
prior_run_id   BIGINT REFERENCES scoring_runs(id), -- anchor for re-scoring
created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE tier_overrides (
  id          BIGSERIAL PRIMARY KEY,
  account_id  BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  scoring_run_id BIGINT NOT NULL REFERENCES scoring_runs(id),
  original_tier TEXT NOT NULL,
  override_tier TEXT NOT NULL,
  reason      TEXT NOT NULL,
  set_by_user_id BIGINT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

\$4.3 Signals

```

CREATE TABLE signals (
  id          BIGSERIAL PRIMARY KEY,
  account_id  BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  signal_type TEXT NOT NULL,         -- leadership_change | funding_round | etc.
  source      TEXT NOT NULL,         -- apollo (V2) | apify_linkedin | crunchbase | google_news | manual
  title       TEXT NOT NULL,
  payload     JSONB NOT NULL,
  url         TEXT,
  observed_at TIMESTAMPTZ NOT NULL,
  processed_at TIMESTAMPTZ,         -- when re-scoring fired (NULL = pending or skipped)
  fingerprint TEXT NOT NULL,       -- dedupe key: hash(source,type,url,observed_date)
  UNIQUE(account_id, fingerprint)
);

CREATE TABLE signal_weights (
  signal_type TEXT PRIMARY KEY,
  base_weight NUMERIC(5,2) NOT NULL,
  half_life_days INTEGER NOT NULL, -- decay parameter
  description TEXT,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_by BIGINT
);

-- Seed values (from GTM team intuition; Galyna can tune later)
-- INSERT: leadership_change 10 180
-- INSERT: ma_announcement   8 365
-- INSERT: ehr_change        8 365
-- INSERT: funding_round     7 270
-- INSERT: layoff_announcement 6 120
-- INSERT: press_release     4 90
-- INSERT: job_posting_rcm    5 60

```

\$4.4 Engagement (the heavy table)

```

-- RAW: every event from APIs, never deleted, sometimes unresolved
CREATE TABLE raw_touch_events (
  id          BIGSERIAL PRIMARY KEY,
  source      TEXT NOT NULL,
  source_event_id TEXT,             -- the API's own ID
  payload     JSONB NOT NULL,
  observed_at TIMESTAMPTZ NOT NULL,
  ingested_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  resolution_status TEXT NOT NULL DEFAULT 'pending',
  -- pending | resolved | unresolved | ignored | manual_required
  resolved_to_account_id BIGINT REFERENCES accounts(id),
  resolved_to_contact_id BIGINT REFERENCES contacts(id),
  resolved_at TIMESTAMPTZ,
  resolved_by TEXT                 -- 'auto:email' | 'auto:linkedin' | 'manual:galyna' | etc.
);

-- RESOLVED: actual engagement touches with weight stamped at insert
CREATE TABLE engagement_touches (
  id          BIGSERIAL PRIMARY KEY,
  account_id  BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  contact_id  BIGINT REFERENCES contacts(id),
  channel     TEXT NOT NULL,       -- BOFU | TOFU | LinkedIn_Content | etc.

```

```

touch_count    INTEGER NOT NULL DEFAULT 1,
content_ref    TEXT,                -- "6 UM Trends 2026", "#1 Vanessa email"
source         TEXT NOT NULL,
raw_event_id   BIGINT REFERENCES raw_touch_events(id), -- backreference
observed_at    TIMESTAMPTZ NOT NULL,
weight_at_insert NUMERIC(5,2) NOT NULL, -- snapshot at insert time
fingerprint    TEXT NOT NULL,
UNIQUE(account_id, fingerprint)
);

CREATE TABLE channel_weights (
  channel      TEXT PRIMARY KEY,
  weight       NUMERIC(5,2) NOT NULL,
  half_life_days INTEGER NOT NULL,    -- engagement decay parameter
  description   TEXT,
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_by    BIGINT
);

-- Seeded from Galyna's xlsx legend:
-- INSERT: BOFU          10 60
-- INSERT: Response_Agreed 10 9999 -- never decay
-- INSERT: TOFU          6 60
-- INSERT: Podcast_Guest  4 90
-- INSERT: Event_Attend   4 90
-- INSERT: Direct_Mail_Response 4 60
-- INSERT: LP_Visit       2 7
-- INSERT: LP_Video_View  2 7
-- INSERT: LinkedIn_Content 2 30
-- INSERT: LinkedIn_Connect 2 30
-- INSERT: Email_Engagement 1 14
-- INSERT: SDR_AE_Cold_Outreach 1 60 -- pending Galyna confirmation

-- BUCKETING STATE: one row per account (denorm cache, refreshed by debounced batch)
CREATE TABLE account_engagement_state (
  account_id    BIGINT PRIMARY KEY REFERENCES accounts(id) ON DELETE CASCADE,
  engagement_total_raw    NUMERIC(8,2) NOT NULL DEFAULT 0,
  engagement_total_decayed NUMERIC(8,2) NOT NULL DEFAULT 0,
  intent_bucket  TEXT,                -- Lower | Some | Warm | Hot
  intent_action  TEXT,                -- resolved from intent_action_rules
  last_recomputed_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  last_touch_at  TIMESTAMPTZ
);

-- DIRTY QUEUE: accounts needing recompute (drained every 60s)
CREATE TABLE dirty_accounts (
  account_id    BIGINT PRIMARY KEY REFERENCES accounts(id) ON DELETE CASCADE,
  marked_dirty_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE intent_thresholds (
  id            BIGSERIAL PRIMARY KEY,
  min_score     NUMERIC(8,2) NOT NULL,
  max_score     NUMERIC(8,2),         -- NULL = open-ended (Hot bucket)
  bucket        TEXT NOT NULL UNIQUE,
  sort_order    INTEGER NOT NULL,
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_by    BIGINT
);

-- Seed: 0-5 Lower | 6-11 Some | 12-20 Warm | 21+ Hot

CREATE TABLE intent_action_rules (
  id            BIGSERIAL PRIMARY KEY,
  bucket        TEXT NOT NULL,
  segment       TEXT,                -- NULL = matches any segment
  status        TEXT,                -- NULL = matches any status
  action_text    TEXT NOT NULL,
  priority      INTEGER NOT NULL,    -- higher priority wins
  active        BOOLEAN NOT NULL DEFAULT TRUE,
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- BUCKET TRANSITION HISTORY (every up/down move logged)
CREATE TABLE account_bucket_transitions (
  id            BIGSERIAL PRIMARY KEY,
  account_id    BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  from_bucket   TEXT,
  to_bucket     TEXT NOT NULL,
  total_at_transition NUMERIC(8,2) NOT NULL,
  triggered_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

§4.5 Ownership, status, notes, audit, users

```

CREATE TABLE users (
  id            BIGSERIAL PRIMARY KEY,
  clerk_user_id TEXT NOT NULL UNIQUE,
  email         TEXT NOT NULL UNIQUE,

```

```

display_name TEXT,
role TEXT NOT NULL DEFAULT 'analyst', -- analyst | owner | admin
is_active BOOLEAN NOT NULL DEFAULT TRUE,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE account_ownership (
id BIGSERIAL PRIMARY KEY,
account_id BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
owner_id BIGINT NOT NULL REFERENCES users(id),
assigned_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
ended_at TIMESTAMPTZ, -- NULL = current
assigned_by_user_id BIGINT REFERENCES users(id),
reason TEXT -- 'initial' | 'transfer' | 'resignation' | 'workload'
);

CREATE TABLE account_status_changes (
id BIGSERIAL PRIMARY KEY,
account_id BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
from_status TEXT,
to_status TEXT NOT NULL,
changed_by_user_id BIGINT REFERENCES users(id),
note TEXT,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE account_notes (
id BIGSERIAL PRIMARY KEY,
account_id BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
user_id BIGINT NOT NULL REFERENCES users(id),
body TEXT NOT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE audit_log (
id BIGSERIAL PRIMARY KEY,
user_id BIGINT REFERENCES users(id), -- NULL for system actions
action TEXT NOT NULL,
entity_type TEXT NOT NULL,
entity_id BIGINT NOT NULL,
details JSONB,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

§4.6 Cost guardrails (6-layer defense)

```

CREATE TABLE scoring_cooldowns (
account_id BIGINT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
cooldown_until TIMESTAMPTZ NOT NULL,
reason TEXT NOT NULL, -- 'hourly_cap' | 'daily_cap' | 'signal_cooldown'
signal_type TEXT, -- for signal_cooldown
PRIMARY KEY(account_id, reason, signal_type)
);

-- Tracking table for global cost guardrails
CREATE TABLE cost_tracking (
id BIGSERIAL PRIMARY KEY,
month DATE NOT NULL, -- first day of month
total_cost_usd NUMERIC(10,4) NOT NULL DEFAULT 0,
total_runs INTEGER NOT NULL DEFAULT 0,
last_updated TIMESTAMPTZ NOT NULL DEFAULT NOW(),
UNIQUE(month)
);

```

The 6 cost layers (locked):

#	Layer	Limit	Behavior on hit
1	Per-account hourly cap	Max 2 scoring runs per account per hour	Refuse new run, insert into <code>scoring_cooldowns</code> until expiry; signal logged but no re-score
2	Per-account daily cap	Max 5 scoring runs per account per 24h	Same as Layer 1 with 24h cooldown
3	Per-signal-type cooldown	Same signal_type for same account within 24h doesn't trigger re-score	Signal still inserted into <code>signals</code> ; <code>processed_at</code> NOT set; logged as "cooldown_skipped"
4	Per-batch pre-flight check	Before enqueueing a batch of N: <code>estimated_cost = N × avg_cost_per_run</code> . If <code>current_mtd + estimated > \$400</code> : require explicit confirmation in UI . If <code>current_mtd + estimated > \$500</code> : refuse the batch entirely	UI shows "This batch will cost ~\$. MTD will reach \$. Confirm/Cancel"

#	Layer	Limit	Behavior on hit
5	Global MTD soft alert	Alert at \$100 MTD	Sentry warning + Slack DM to admin (reason='cost_soft_alert')
6	Global MTD hard cap	Refuse new scoring jobs above \$500 MTD	All scoring endpoints return 429 with clear error: "monthly budget reached"; manual unblock via COST_OVERRIDE=1 env var

Computation of `avg_cost_per_run` : rolling 7-day average from `scoring_runs.cost_usd` . Updates daily. Fallback to \$0.30 if no prior data (cold start).

Pre-flight check implementation:

```
async def estimate_batch_cost(num_accounts: int) -> BatchCostEstimate:
    avg = await get_rolling_avg_cost_per_run(days=7) or 0.30
    estimated = num_accounts * avg
    mtd = await get_mtd_cost()
    return BatchCostEstimate(
        num_accounts=num_accounts,
        estimated_usd=estimated,
        mtd_current_usd=mtd,
        mtd_projected_usd=mtd + estimated,
        requires_confirmation=(mtd + estimated > 400),
        will_be_refused=(mtd + estimated > 500),
    )
```

§4.7 The "effective state" views (Flaw 9 fix)

Two views, each with a distinct purpose. Picking the wrong one in a list query returns duplicate rows for multi-segment accounts.

```
-- VIEW A -- ONE ROW PER ACCOUNT (for "show me all accounts" list views)
-- Picks the highest-scored segment as the representative.
-- Use this for: Galyna's Triage, Owner's Pipeline, generic account lists.
CREATE VIEW accounts_view AS
SELECT DISTINCT ON (a.id)
    a.*,
    s.segment AS primary_segment,
    COALESCE(o.override_tier, sr.tier) AS effective_tier,
    sr.total_score AS effective_score,
    sr.score_band_low,
    sr.score_band_high,
    sr.max_score AS effective_max,
    sr.created_at AS effective_last_scored_at,
    es.engagement_total_decayed AS engagement_total,
    es.intent_bucket,
    es.intent_action,
    cur_own.owner_id AS current_owner_id,
    cur_own.assigned_at AS current_owner_since,
    (SELECT COUNT(*) FROM account_segments WHERE account_id = a.id) AS segment_count
FROM accounts a
JOIN account_segments s ON s.account_id = a.id
LEFT JOIN LATERAL (
    SELECT * FROM scoring_runs
    WHERE account_id = a.id AND segment = s.segment
    ORDER BY created_at DESC LIMIT 1
) sr ON TRUE
LEFT JOIN LATERAL (
    SELECT * FROM tier_overrides
    WHERE account_id = a.id AND scoring_run_id = sr.id
    ORDER BY created_at DESC LIMIT 1
) o ON TRUE
LEFT JOIN account_engagement_state es ON es.account_id = a.id
LEFT JOIN LATERAL (
    SELECT * FROM account_ownership
    WHERE account_id = a.id AND ended_at IS NULL
    LIMIT 1
) cur_own ON TRUE
ORDER BY a.id, s.current_score DESC NULLS LAST;

-- VIEW B -- ONE ROW PER (ACCOUNT, SEGMENT) -- for segment-scoped queries
-- Example: "Show me all High Fit specialties accounts"
-- Use this for: per-segment filters, per-segment cards on Account Detail.
CREATE VIEW account_segments_view AS
SELECT
    a.*,
    s.segment,
    COALESCE(o.override_tier, sr.tier) AS effective_tier,
    sr.total_score AS effective_score,
    sr.score_band_low,
    sr.score_band_high,
    sr.max_score AS effective_max,
    sr.created_at AS effective_last_scored_at,
    es.engagement_total_decayed AS engagement_total,
    es.intent_bucket,
    es.intent_action,
```

```

    cur_own.owner_id AS current_owner_id,
    cur_own.assigned_at AS current_owner_since
FROM accounts a
JOIN account_segments s ON s.account_id = a.id
LEFT JOIN LATERAL (
    SELECT * FROM scoring_runs
    WHERE account_id = a.id AND segment = s.segment
    ORDER BY created_at DESC LIMIT 1
) sr ON TRUE
LEFT JOIN LATERAL (
    SELECT * FROM tier_overrides
    WHERE account_id = a.id AND scoring_run_id = sr.id
    ORDER BY created_at DESC LIMIT 1
) o ON TRUE
LEFT JOIN account_engagement_state es ON es.account_id = a.id
LEFT JOIN LATERAL (
    SELECT * FROM account_ownership
    WHERE account_id = a.id AND ended_at IS NULL
    LIMIT 1
) cur_own ON TRUE;

```

Usage rule:

- `accounts_view` — list views that should show one row per company (Triage, Owner Pipeline, etc.)
- `account_segments_view` — segment-filtered queries and per-segment Account Detail panels
- Replace any old reference to `accounts_with_state` in routers/services with the correct one

§4.8 Indexes

```

-- Hot query paths
CREATE INDEX idx_accounts_status      ON accounts(current_status);
CREATE INDEX idx_accounts_activated   ON accounts(is_activated) WHERE is_activated;
CREATE INDEX idx_accounts_suppressed  ON accounts(is_suppressed) WHERE is_suppressed;
CREATE INDEX idx_accounts_parked      ON accounts(parked_until) WHERE current_status='PARKED';

CREATE INDEX idx_segments_account     ON account_segments(account_id);
CREATE INDEX idx_segments_tier        ON account_segments(segment, current_tier);

CREATE INDEX idx_contacts_account     ON contacts(account_id);
CREATE INDEX idx_contacts_email       ON contacts(LOWER(email));
CREATE INDEX idx_contacts_linkedin    ON contacts(linkedin_url);

CREATE INDEX idx_signals_account_obs  ON signals(account_id, observed_at DESC);
CREATE INDEX idx_signals_pending      ON signals(processed_at) WHERE processed_at IS NULL;

CREATE INDEX idx_raw_pending          ON raw_touch_events(resolution_status)
                                     WHERE resolution_status='pending';
CREATE INDEX idx_raw_unresolved       ON raw_touch_events(resolution_status)
                                     WHERE resolution_status='unresolved';
CREATE INDEX idx_engagement_account_obs ON engagement_touches(account_id, observed_at DESC);
CREATE INDEX idx_engagement_channel   ON engagement_touches(channel);

CREATE INDEX idx_runs_account_created ON scoring_runs(account_id, created_at DESC);
CREATE INDEX idx_runs_segment         ON scoring_runs(segment, created_at DESC);
CREATE INDEX idx_runs_failed          ON scoring_runs(parse_failed) WHERE parse_failed;

CREATE INDEX idx_dirty_marked         ON dirty_accounts(marked_dirty_at);

CREATE INDEX idx_status_changes_account ON account_status_changes(account_id, created_at DESC);

-- Partial unique index: only ONE current (un-ended) ownership per account
CREATE UNIQUE INDEX one_current_owner_per_account
    ON account_ownership(account_id) WHERE ended_at IS NULL;

CREATE INDEX idx_audit_user_action    ON audit_log(user_id, action, created_at DESC);
CREATE INDEX idx_audit_entity         ON audit_log(entity_type, entity_id);

CREATE INDEX idx_cooldowns_until      ON scoring_cooldowns(cooldown_until);

```

§5 — The connector framework

```

# All input sources (account / signal / engagement) implement one protocol.

class Connector(Protocol):
    name: str # 'definitive' | 'reply_io' | etc.
    kind: Literal['account', 'signal', 'engagement']
    schedule: str | None # cron-ish, e.g. '*/15 * * * *' or None for webhook-only

    async def poll(self, since: datetime) -> AsyncIterator[Record]: ...
    async def handle_webhook(self, payload: dict, sig: str) -> AsyncIterator[Record]: ...

class Record(BaseModel):
    source_event_id: str | None # the API's own ID for idempotency
    payload: dict # full source-specific blob (JSONB)
    observed_at: datetime

```



```
# For account connectors: hints to identify the account
account_match_hint: dict | None          # {'name': ..., 'domain': ..., 'ein': ...}

# For signal/engagement connectors: hints to identify the contact
contact_match_hint: dict | None          # {'email': ..., 'linkedin_url': ..., 'name': ...}

# Each connector is a separate file in apps/api/src/abm_api/connectors/
# e.g. apps/api/src/abm_api/connectors/reply_io.py
```

Routing: all incoming records go into `raw_touch_events` (engagement) or directly into `signals / accounts` (signal/account connectors). Identity resolution happens as a separate batch job for engagement.

Failure isolation: each connector has its own Arq queue with retries (3, exponential backoff) and dead-letter handling.

Connector scope (V1 vs V2):

Source	Role	Stage
Reply.io	Email engagement (opens, clicks, replies)	V1 — first engagement connector
Apollo	Contact enrichment + leadership-change signals (not email engagement)	V2

Do not use Apollo for email engagement in docs or code — production email is Reply.io.

§6 — Tech stack (final picks)

Layer	Tech	Why
Frontend	Next.js 15 (App Router) + TypeScript + Tailwind + shadcn/ui	Industry standard; pairs with Claude Design
Backend API	FastAPI + Pydantic v2	Async-native, OpenAPI-first, where the scoring code already lives
Database	Postgres 16 (Railway-managed)	Mid-scale fits; portable to Snowflake later
Migrations	Alembic	Standard with Pydantic
Cache + Queue	Redis (Railway-managed)	Arq backing store
Background jobs	Arq	Async-native, Pydantic-friendly, much lighter than Celery
Connector framework	Custom protocol + Arq jobs per connector	One pattern for all input sources
Auth	Clerk (magic link + Google SSO)	Multi-user ready
LLM	Anthropic Claude Opus 4.7 + <code>web_search_20260209</code>	Validated
Hosting	All on Railway (single project, multi-service)	One vendor
Observability	Sentry + structlog JSON to Railway log drain	Free at V1 scale
Python tooling	uv + ruff + mypy + pytest	Astral stack
JS tooling	pnpm + vitest + playwright + biome	Rust toolchain
CI	GitHub Actions	Matrix lint/type/test/build
PDF generation	weasyprint (server-side, no Chrome)	Production reliability

§7 — The 15 architectural decisions (final, with rationale)

#	Decision	Status	Why
1	<code>accounts</code> = entity, <code>account_segments</code> = scoring lens (multi-segment supported)	 Locked	Real entities (Ascension, Optum) span segments. Avoids duplicates. Honest attribution.

#	Decision	Status	Why
2	Append-only <code>engagement_touches</code> + debounced 60s recompute via <code>account_engagement_state</code> and <code>dirty_accounts</code> queue	✓ Locked	Scales to 10M+ touches without deadlocks. One Slack DM per burst, not 30.
3	2-stage ingestion: <code>raw_touch_events</code> → identity resolver → <code>engagement_touches</code> ; unresolved kept for manual review	✓ Locked	30-60% of engagement signal preserved that would otherwise be lost. Audit trail.
4	Anchored re-scoring (inject prior context into Claude prompt) + score bands in UI	✓ Locked	Eliminates "score swings randomly" trust problem. Honest uncertainty.
5	Suppression: scored & tracked, never activated/alerted until cleared. <code>is_suppressed</code> + <code>suppressed_until</code>	✓ Locked	Legal protection + CS team can block during renewals.
6	<code>weight_at_insert</code> stamped per touch; channel weight changes apply forward only	✓ Locked	History immutable. Galyna can tune without breaking reports.
7	<code>account_ownership</code> table with full history (assigned_at, ended_at, reason, assigned_by)	✓ Locked	Audit trail when SDRs leave/rotate. Honest attribution.
8	Signal decay with per-signal-type half-lives in <code>signal_weights</code> table	✓ Locked	Fresh signals dominate. Old signals fade. Matches human intent perception.
9	Drop denorm <code>current_*</code> cols; compute via <code>accounts_view</code> (one row per account) and <code>account_segments_view</code> (one row per account+segment) — see §4.7	✓ Locked	One source of truth. No cache-invalidation bugs. Split views prevent duplicate-row bugs for multi-segment accounts.
10	<code>intent_action_rules</code> engine matching on (bucket, segment, status, priority)	✓ Locked	Actions match real-world playbooks per segment per stage.
11	Audit log: 2 years hot in <code>audit_log</code> ; archive to cold storage (V1.5)	⏸ Deferred V1.5	Partitioning only when >10M rows. V1 scale doesn't need it.
12	<code>tenant_id</code> from day 1	⏸ Deferred V2	Known migration debt; flagged in code comments.
13	<code>structured_json_version</code> column + version-aware Pydantic readers	✓ Locked	Schema evolution without breaking old data.
14	V1 status chain: <code>COLD</code> → <code>ACTIVATED</code> → <code>MEETING_BOOKED</code> → <code>MEETING_HELD</code> + <code>PARKED</code> . V2 SFDC handoff for <code>QUALIFIED/WON/LOST</code>	✓ Locked	V1 ships without SFDC dependency. SFDC takes over post- <code>MEETING_HELD</code> in V2.
15	Cost guardrails (6 layers, see §4.6): per-account hourly cap (2/h) + per-account daily cap (5/d) + per-signal cooldown (24h) + per-batch pre-flight check (confirm >\$400, refuse >\$500 projected MTD) + global soft alert (\$100 MTD) + global hard cap (\$500 MTD)	✓ Locked	Multi-layer defense. Batch pre-flight catches large manual runs before they bankrupt. Signal storms can't bankrupt either.

§8 — The 16 engineering decisions (final, with rationale)

#	Question	Decision
1	Fuzzy match threshold for duplicate company names	≥95% auto-merge / 80-95% review / <80% separate. Normalize first (lowercase, strip LLC/Inc/Health System), t
2	Definitive + manual upload dedupe priority	(1) domain exact → (2) normalized_name exact (entity-level, no segment in this key) → (3) fuzzy review queue p When a row arrives with a new segment for an existing entity, INSERT an account_segments row — never c
3	Contact moves companies (Matt Avance Care → OrthoIndy)	New contact at new account; old contact gets ended_at ; engagement stays with old contact (historical truth). C
7	Same email open emits N events	Once per (contact, email_id, day) — debounce in resolver
9	Engagement decay per channel	Galyna-tunable. Defaults: LinkedIn 30d / Email 14d / LP 7d / Podcast & Event 90d / Response_Agreed never
14	Score drops High→Med on re-score; activation status?	Stay activated, show "Score dropped" badge
16	Cohort definition for reports	Pipeline reports = activation date. Scoring-quality reports = scoring date. Both available; right one is default per
17	Conversion attribution	Multi-touch 40/30/30 (first 40% / middle 30% / last 30%)

#	Question	Decision
18	Reports raw vs decayed	Decayed by default + toggle for raw cumulative
19	V1 owner visibility	All owners see all accounts; default filter "my accounts". V2 strict per-owner.
21	Audit retention	2 years hot, then cold archive to S3/R2
22	Connector down detection	warn 3h / Sentry 12h / Slack DM 24h. <code>last_successful_poll_at</code> per connector. Admin UI shows real-time health
23	Claude parse failure / refusal	Save with <code>parse_failed=TRUE</code> + <code>raw_markdown</code> ; account score stays at prior value; UI flags "last failed"
24	Concurrent edits	Optimistic locking with <code>version</code> column; 409 on mismatch; client refreshes + retries
25	PDF storage	Regenerate on demand from <code>raw_markdown</code> (weasyprint ~3s); no caching at V1 scale
26	HIPAA relevance	Not HIPAA-relevant in V1. Document in DECISIONS.md. Re-evaluate if a connector ingests PHI-adjacent data later

§9 — The 9 workflow questions for Galyna

These don't block engineering work — V1 can be built without them — but they need her decisions before activation flow polish and reporting build.

#	Question	What it unblocks	Default if she doesn't answer
Q4	For multi-segment accounts (Ascension), owner per segment or global owner?	Multi-segment account UI design	Global owner (simpler)
Q4a	For corporate parents with regional subsidiaries (Ascension Wisconsin, Ascension Florida, Ascension Texas, etc.) — model as ONE account (Ascension) with multiple segments, or as SEPARATE accounts with a <code>parent_account_id</code> link?	Dedupe rule: domain match collapses by default. If "separate" is preferred, need a <code>parent_account_id</code> column on <code>accounts</code> and explicit parent-child UI.	Collapse by default (UNIQUE(domain) handles it). Add <code>parent_account_id</code> only if Galyna explicitly wants subsidiary isolation.
Q5	For multi-segment accounts, engagement counts globally or per-segment?	Engagement attribution for multi-segment	Global (counted toward the whole account)

#	Question	What it unblocks	Default if she doesn't answer
Q6	Activation per segment or per account? Can she activate Ascension-HS while Specialties stays cold?	Activation UI design	Per account (activate the whole entity)
Q10	What happens to Colin's accounts when he's out 2 weeks?	Out-of-office workflow	Manual reassign by Galyna (no auto-redistribute)
Q11	Two owners per account (primary + secondary)?	SDR/AE pairing workflows	Single owner only
Q12	Workload balancing — refuse 50th account when owner has 40 active?	Activation UI behavior	Warn but don't refuse
Q13	Score freshness — at what age is a score "stale"? Auto-rescore at that age?	Stale-account UI badge + scheduled re-score job	60 days stale; auto-rescore for activated accounts only
Q20	Can SDR Colin override tier on his own accounts? Or only Galyna?	Permission model in tier override UI	Only Galyna (V1 single power user)

Recommendation: route these to Galyna in a single async DM with a 1-page brief. Don't block Phase 1 build on her response.

§10 — Failure mode catalog

Every documented failure mode and its locked behavior:

Failure	Locked behavior
Definitive API down	Connector logs error; retries indefinitely with exponential backoff. UI admin panel shows "last successful poll N hours ago."
Reply.io API down	Same pattern; engagement loop pauses for that source; other sources continue.
LinkedIn API down	Same pattern.
gan.ai analytics API down	Same pattern.
Claude API 429	SDK auto-retries with backoff (built-in).
Claude API 5xx	Arq job retries 3x with exponential backoff; on persistent failure, save with <code>parse_failed=TRUE</code> .
Claude returns garbage JSON	<code>parse_failed=TRUE</code> , <code>raw_markdown</code> preserved, account score stays at prior value. UI flags "last failed."
Claude refusal	<code>stop_reason='refusal'</code> ; alert Galyna; account unchanged.
Postgres slow	Connection pool exhaustion; reads degrade gracefully (views work, writes queue).
Redis down	Arq jobs cannot enqueue; FastAPI returns 503 for /score endpoints; reads still work.
Identity resolver overwhelmed	Raw events stay in <code>raw_touch_events</code> ; backlog visible in admin UI.
Engagement recompute lagging	UI shows "engagement last updated N minutes ago" on account detail.
Cost soft alert (\$100 MTD)	Sentry warning; Slack DM to admin.
Cost hard cap (\$500 MTD)	New scoring jobs refused with clear error message; existing dirty queue continues. Manual unblock via env var.
Batch pre-flight (>\$400 projected MTD)	UI prompts user: "This batch will cost ~\$X. MTD will reach \$Y. Confirm/Cancel." Galyna must click Confirm to enqueue.
Batch pre-flight (>\$500 projected MTD)	Batch refused before enqueue. Error: "Monthly budget would be exceeded by this batch." Galyna can split the batch or wait for next month.
Per-account hourly cap	Excess scoring requests refused with "cooldown active until X"; signal logged but not actioned.
Identity resolution stuck	Manual UI for Galyna to assign account → contact mapping.
Account version conflict (Q24)	409 to client; UI auto-refreshes + retries.
Connector authentication failure	Single Sentry alert; connector disabled; admin notified.
Webhook signature failure	Reject with 401; log incident; do not insert event.

§11 — Phase plan (concrete, sequenced)

#	Phase	Effort	Outcome	Locked Decisions Implemented
1	Foundation refactor	2 sessions	Monorepo (<code>apps/api</code> , <code>apps/web</code> , <code>packages/shared-types</code>), Pydantic v2 models for all tables, Postgres schema via Alembic, Arq worker, structured logging, scoring engine lifted from CLI	1, 2 (schema), 3 (schema), 4 (anchor prompt), 5 (schema), 6, 7, 8, 9, 12 (deferred docs), 13, 14 (V1 chain), 15
2	Web app skeleton + auth + Sentry	2 sessions	Next.js + Clerk + FastAPI + monorepo CI + Sentry + Railway deploy	(Infrastructure only)
3	Universe + Scoring + Score-New UI	3 sessions	Definitive API connector, CSV upload, Triage view, Score-New form with live streaming, Account Detail	1 (active queries), 2 (active recompute), 4 (UI bands), 9 (view-based queries), 23
4	Activation + Owner routing + Slack	1 session	Activation flow, owner assignment, status transitions, Slack DMs on activation + bucket changes	5, 7, 14
5	Engagement: Reply.io (first connector)	2 sessions	First engagement connector, <code>raw_touch_events</code> , identity resolver, <code>engagement_touchees</code> with weight stamping	2, 3, 6, 8 (decay), 9
6	Engagement: LinkedIn + gan.ai + LP analytics	3 sessions	Three connectors in parallel	22 (health), 23 (failure)
7	Funnel + Reporting (P0)	2 sessions	Status transitions, owner pipeline view, attribution dashboard, multi-touch attribution, cost dashboard	16, 17, 18
8	Polish + ops + RUNBOOK	1 session	Cost dashboard tile, backup verification, RUNBOOK, monitoring alerts, V2 handoff	15 (cost dashboard), 19, 21

Total: ~16 sessions to ship V1-complete.

§12 — Production-grade requirements

Category	Requirement	Tool / pattern
Repo structure	Monorepo: <code>apps/api</code> + <code>apps/web</code> + <code>packages/shared-types</code>	pnpm workspaces + uv
Type safety	Pydantic v2 at every API boundary; OpenAPI → TS client; strict mode	openapi-typescript
Testing	Unit (pytest + vitest), integration (httpx + testcontainers Postgres), E2E (playwright smoke)	Coverage ≥ 70% on core domain
Lint + format	Ruff (Python), Biome (JS/TS); both pre-commit + CI	ruff, biome
Type checking	mypy strict; tsc strict	CI gate
Pre-commit	Hooks for ruff, mypy, biome, secret-scan	pre-commit
CI/CD	GitHub Actions: lint → type → test → build on PR; auto-deploy main → Railway	Branch protection
DB migrations	Alembic with autogenerate + manual review; one migration per PR	Required check
Secrets	Railway env vars (V1); rotation runbook	Annual rotation reminder
Auth	Clerk middleware on every API route except <code>/health</code> , <code>/ready</code> ; audit on state changes	Clerk + middleware
Error tracking	Sentry on api + web + worker	Release tagging
Structured logging	JSON with trace IDs (W3C traceparent)	structlog + pino
Rate limiting	Per-endpoint (<code>fastapi-limiter</code>); per-account scoring cooldowns; MTD cost cap	Multi-layer
API versioning	<code>/api/v1/...</code> from day 1	Routes namespaced
Background jobs	Arq: retries (3, exponential backoff), dead-letter, deterministic job IDs	Per-connector queue

Category	Requirement	Tool / pattern
Idempotency	All mutating endpoints accept <code>Idempotency-Key</code> ; engagement deduped by fingerprint	Middleware
Health + readiness	<code>/health</code> (cheap), <code>/ready</code> (DB + Redis + Anthropic check)	Railway healthcheck
Feature flags	Env-var-driven flag dict	Custom
DB backups	Railway managed daily snapshots, 7-day retention	Documented
Cost guardrails	6-layer (see §4.6)	Custom + Sentry
Optimistic locking	<code>version</code> column on <code>accounts</code> , 409 on mismatch	App-level middleware
Docs	OpenAPI auto-generated; README + RUNBOOK + DECISIONS (ADRs)	One ADR per non-obvious choice
Repo hygiene	Conventional commits; squash-merge; release-please for CHANGELOG	Automated

§13 — Repo layout (final)

```

abm-scorer/
├── pnpm-workspace.yaml
├── pyproject.toml
├── .github/workflows/
│   ├── ci.yml
│   └── deploy.yml
├── .pre-commit-config.yaml
├── .env.example
├── README.md
├── apps/
│   ├── api/
│   │   ├── pyproject.toml
│   │   ├── alembic/
│   │   └── src/abm_api/
│   │       ├── main.py
│   │       ├── config.py
│   │       ├── db.py
│   │       └── models/
│   │           ├── accounts.py
│   │           ├── segments.py
│   │           ├── contacts.py
│   │           ├── signals.py
│   │           ├── engagement.py
│   │           ├── scoring.py
│   │           └── audit.py
│   │       ├── routers/
│   │       │   ├── accounts.py
│   │       │   ├── scoring.py
│   │       │   ├── signals.py
│   │       │   ├── engagement.py
│   │       │   ├── reports.py
│   │       │   └── webhooks/
│   │       │       ├── reply_io.py
│   │       │       └── gan_ai.py
│   │       ├── services/
│   │       │   ├── scoring_engine.py
│   │       │   ├── identity_resolver.py
│   │       │   ├── engagement_recompute.py
│   │       │   ├── intent_actions.py
│   │       │   ├── decay.py
│   │       │   └── cost_guardrails.py
│   │       ├── connectors/
│   │       │   ├── base.py
│   │       │   ├── definitive.py
│   │       │   ├── reply_io.py
│   │       │   ├── linkedin_sales_nav.py
│   │       │   ├── gan_ai_analytics.py
│   │       │   └── lp_analytics.py
│   │       ├── workers/
│   │       │   ├── arq_app.py
│   │       │   ├── score_job.py
│   │       │   ├── identity_resolver_job.py
│   │       │   ├── engagement_recompute_job.py
│   │       │   └── connector_poll_job.py
│   │       ├── prompts/
│   │       │   ├── specialties.txt
│   │       │   ├── payers.txt
│   │       │   └── health_systems.txt
│   │       └── output.py
│   └── tests/
├── web/
├── package.json
└── app/

```

```
├── (auth)/sign-in
├── (app)/triage
├── (app)/accounts/[id]
├── (app)/score-new
├── (app)/admin/           # connector health, cost dashboard
├── (app)/settings/        # weights, thresholds, action rules
├── api/                   # Next.js route handlers
├── components/           # shadcn/ui + custom
├── lib/                   # api client generated from OpenAPI
├── tests/
├── worker/               # Arq worker entry
├── packages/
├── shared-types/         # generated TS types from API OpenAPI
├── docs/
├── FINAL_ARCHITECTURE.md  # this file
├── RUNBOOK.md            # incident response, common ops
├── DECISIONS.md          # ADRs
├── scripts/
├── seed_demo_accounts.py
├── backup_db.sh
├── replay_audit_archive.py
└── .env.example
```

§14 — Scale targets (V1)

Metric	V1 target	What breaks first if exceeded
Accounts	5,000	None — UI filter UX
Contacts	50,000	None
Signals	100,000	Add partitioning if >500K
Scoring runs	50,000	Partitioning by month if >200K
Engagement touches	5,000,000	Partitioning by month if >1M
Raw touch events	50,000,000	Archive after 30 days
Audit log	10,000,000	Partition (V1.5 work)
Concurrent users	20	Far below limits
Scoring jobs/day	200	Anthropic rate limits hit first
Engagement events/sec	50 sustained, 500 burst	Recompute debounce critical

If any exceeded: scale up (Railway plan, bigger Postgres, more workers). Architecture unchanged.

§15 — Glossary

- **Account** — a company we target. One row per legal entity in `accounts`.
- **Account segment** — a scoring lens applied to an account. 1+ rows per account in `account_segments`.
- **Contact** — a person at an account. Multiple per account.
- **Signal** — an externally-observed event (leadership change, M&A, funding).
- **Engagement touch** — a first-party event (LinkedIn engagement, email open, LP visit).
- **Raw touch event** — incoming event before identity resolution.
- **Scoring run** — one Claude execution. Immutable, versioned.
- **Tier** — Claude's fit verdict (High Fit / Tier 1 / etc.) — segment-dependent.
- **Score band** — uncertainty range around a score ("25 (24-26)").
- **Intent bucket** — engagement-driven categorization (Lower / Some / Warm / Hot).
- **Intent action** — recommended next step per (bucket, segment, status).
- **Activation** — Galyna's commit to pursue. Assigns Owner. Logs `account_ownership`.
- **Owner** — current SDR/AE assigned. Historical chain in `account_ownership`.
- **Funnel status** — `COLD` → `ACTIVATED` → `MEETING_BOOKED` → `MEETING_HELD` → `PARKED` (V1).
- **Connector** — pluggable input module (account/signal/engagement source).
- **Decay** — exponential discount applied to age-old signals + engagement.
- **Suppression** — account is scored & tracked but never activated or alerted.
- **Triage** — Galyna's review surface (what's new this week).

§16 — ADRs to write (one per non-obvious decision)

Short ADRs (one page each) to commit so future engineers understand the rationale:

- 1. Account vs Segment split — entity model
- 2. Append-only engagement + debounced recompute — scale pattern
- 3. Identity resolution as 2-stage — data integrity
- 4. Anchored re-scoring + bands — Claude determinism
- 5. Suppression as first-class — legal + CS
- 6. weight_at_insert — history immutability
- 7. account_ownership history — audit + attribution
- 8. Signal decay per type — intent realism
- 9. SQL view for effective state — cache-invalidation avoidance
- 0. intent_action_rules engine — playbook flexibility
- 1. Audit log retention 2yr hot — compliance + perf
- 2. tenant_id deferred to V2 — known migration debt
- 3. structured_json versioning — schema evolution
- 4. V1 status chain + V2 SFDC handoff — staged complexity
- 5. Cost guardrails (6 layers, incl. per-batch pre-flight) — defense in depth
- 6. Fuzzy match thresholds — duplicate handling
- 7. Contact career history — engagement attribution
- 8. Engagement decay per channel — channel reality
- 9. Multi-touch attribution 40/30/30 — fair credit
- 0. PDFs regenerated on demand — storage minimalism
- 1. Not HIPAA-relevant in V1 — defensive deferral

§17 — What "done" looks like (sliced)

V1 is shipped in **three named slices**. Each is a real, deployable, demoable build. Don't conflate them when scoping or estimating.

§17.1 V1.0 LAUNCH SLICE (~4 weeks — the demo / first-ship target)

The smallest thing worth showing Galyna in production. Everything required for her to use the platform for her current weekly workflow, even with engagement still on her spreadsheet.

Capability	Done when
Universe building (manual + CSV)	CSV upload for Becker's/PRI/APTA works; manual UI add works; dedupe + fuzzy review queue functional. Definitive API connector is V1.1 if access delayed.
Scoring	Galyna can score any new account via Score-New form; report renders with bands; PDF downloads; re-score uses anchoring; structured JSON validates
Asset linking	Galyna can attach LP url + video url + script per account; no generation
Activation + owner routing	Activation modal works; account moves from COLD → ACTIVATED; owner assigned; Slack DM fires to owner
Suppression	Galyna can mark account as suppressed with reason + until-date; suppressed accounts excluded from triage + alerts
Auth + audit	Clerk works (magic link + Google SSO); audit_log captures every state-changing action
Deployment	Postgres + Redis + FastAPI + Next.js + Arq worker deployed to Railway; Sentry catching errors; CI runs on every PR
Cost basics	MTD spend visible in admin UI; soft alert at \$100 fires; hard cap at \$500 refuses new jobs

Out of V1.0: engagement connectors, intent bucketing, Slack DMs on bucket changes, reporting/attribution dashboards, batch pre-flight check, status transitions past ACTIVATED.

§17.2 V1.1 SLICE (~4 weeks after V1.0)

Engagement loop comes online. Galyna stops maintaining her spreadsheet.

Capability	Done when
Definitive API connector	Nightly pull seeds accounts; first-100 dedupe verified without false positives
Engagement (Reply.io first)	Reply.io webhook + poll active; raw_touch_events ingested; identity resolver runs every 5 min; manual review queue UI functional
Intent bucketing	Debounce 60s recompute runs; weight_at_insert stamped per touch; bucket transitions visible in UI
Bucket transition Slack DMs	Owner gets DM on Warm→Hot transition; no spam on rapid touches (debounced)
Status transitions	Owner can move ACTIVATED → MEETING_BOOKED → MEETING_HELD; transitions audit-logged; notes append
Cost batch pre-flight	Layer 4 active: batch enqueue UI prompts for confirm/refuse based on projected MTD

Capability	Done when
Failure handling	All §10 connector + Claude failure modes tested in staging

§17.3 V1.2 SLICE (~4 weeks after V1.1)

Reporting + remaining connectors. Leadership dashboard goes live.

Capability	Done when
More engagement connectors	LinkedIn Sales Nav + gan.ai analytics + LP analytics all active
Triage dashboard	"What's new this week" view live for Galyna with stats hero + activity feed + filters
Owner pipeline view	Each owner sees their accounts ranked by bucket + days since last touch
Leadership attribution dashboard	Source → conversion view; multi-touch 40/30/30 attribution; cost-per-scored-account; ARR attribution
Cost dashboard tile	Full §4.6 dashboard live; cooldown visibility; anomaly trends
RUNBOOK	Incident response procedures documented for every §10 failure mode

Total: ~12 weeks from kickoff to V1-complete. The phase plan in §11 splits into ~16 sessions; mapped to slices:

- V1.0 = Phases 1, 2, 3 (foundation, web skeleton, partial universe+scoring UI)
- V1.1 = Phases 4, 5 (activation + first engagement connector)
- V1.2 = Phases 6, 7, 8 (more connectors, reporting, polish)

§18 — V2 roadmap (post-V1)

Item	Why deferred from V1	Trigger to revisit
SFDC integration (read/write)	API access pending	Within 90 days of V1 ship
Multi-tenant (tenant_id active)	YAGNI right now; known migration debt	If Magical white-labels or splits internally
Audit log partitioning	V1 scale doesn't need it	When <code>audit_log</code> >10M rows
Strict per-owner permissions	V1 single power user	When SDRs/AEs onboard as primary users
Outbound automation (auto-send emails on activation)	Decision support boundary	When scoring + engagement validate trust
Contact-level engagement tracking	API restrictions; volume concerns	When contact-specific outreach is needed
A/B experimentation framework	Insufficient conversion volume	When 12+ months of conversion data accumulated
Snowflake migration	Postgres sufficient at V1 scale	When >50K accounts or >50M touches
Per-segment owners	Galyna decides if multi-segment workflow demands this	When multi-segment accounts >5% of total
Two-owner accounts (SDR+AE pairing)	Galyna decides	If team adopts pairing model
Notion auto-push	App is source of truth in V1	If marketing wants public-facing artifacts
LP + Video generation automation	Manual pipeline works today	When manual generation becomes the bottleneck
Triple-sampling scoring	Cost-prohibitive at V1	If anchored re-scoring + bands prove insufficient

§19 — Next steps

1. **Galyna sign-off** on the 9 workflow questions (§9) — async DM with 1-pager
2. **Send Galyna this doc** for architectural review
3. **Start Phase 1 implementation** — Foundation refactor
4. **Create GitHub issues** from the phase plan (§11) — one issue per phase, sub-issues per step
5. **Write the 21 ADRs** (§16) one at a time as decisions are implemented
6. **Brief Claude Design** with the §3 journey + §9 workflow questions for screen mocks

This document is the lock-in. Future architectural changes should be ADRs that modify a section here, not rewrites of the document.